

AI-style Block World Solver

```
# 🧠 AI-style Block World Solver

# Starting positions of blocks
start_state = {
    "A": "B",
    "B": "Table",
    "C": "Table"
}

# Goal positions of blocks
goal_state = {
    "C": "B",
    "B": "A",
    "A": "Table"
}

# This function tries to reach the goal using step-by-step logic
def solve_block_world_ai(start, goal):
    # Show start state
    print("📍 Start State:")
    for block, position in start.items():
        print(f"  {block} is on {position}")

    # Show goal state
    print("\n🎯 Goal State:")
    for block, position in goal.items():
        print(f"  {block} should be on {position}")

    print("\n🧠 AI Thinking and Moving:\n")

    # Let's track current state (like AI's memory)
    current = start.copy()
    steps = 1

    # We'll loop until current matches goal
    while current != goal:
        for block in goal:
            # Check if this block is already in correct position
            if current[block] != goal[block]:
                # Make a move (like AI decides to do something)
                print(f"Step {steps}: Move {block} from {current[block]} to {goal[block]}")
                current[block] = goal[block]  # Update the AI's world
                steps += 1
                break  # Only move one block at a time (simple logic)

    print("\n🎉 Goal Reached!")
```

AI-style Block World Solver

```
# Show final state
print("\n📦 Final State:")
for block, position in current.items():
    print(f"    {block} is on {position}")

# Run our simple AI
solve_block_world_ai(start_state, goal_state)
```

Output

[Clear](#)

📌 Start State:

A is on B
B is on Table
C is on Table

🎯 Goal State:

C should be on B
B should be on A
A should be on Table

🤖 AI Thinking and Moving:

Step 1: Move C from Table to B
Step 2: Move B from Table to A
Step 3: Move A from B to Table

✅ Goal Reached!

📦 Final State:

A is on Table
B is on A
C is on B

=== Code Execution Successful ===